PART I: BUILDING 3D SHAPES

1. A `Shape3D` leaf node builds a 3D object with:
   - Geometry:
     - The form or structure of a shape
   - Appearance:
     - The coloration, transparency, and shading of a shape
2. The `Shape3D` class extends the `Leaf` class

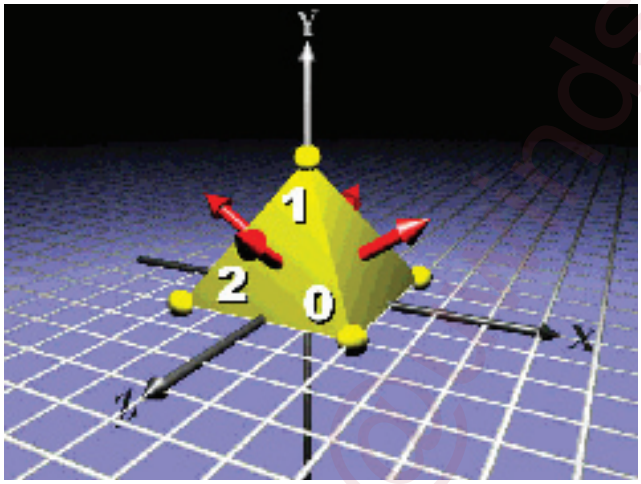
Class Hierarchy
java.lang.Object └ org.jogamp.java3d.SceneGraphObject └ org.jogamp.java3d.Node └ org.jogamp.java3d.Leaf └ org.jogamp.java3d.Shape3D

3. `Shape3D` methods to set geometry/appearance attributes

Method
`Shape3D( )`
`Shape3D( Geometry geometry, Appearance appearance )`
`void setGeometry( Geometry geometry )`
`void setAppearance( Appearance appearance )`

4. Building geometry using coordinates
   - Using a right-handed coordinate system
     - The right-handed coordinate system
       - X = left-to-right
       - Y = bottom-to-top
       - Z = back-to-front
     - Distances are conventionally in meters.

- Building geometry is like a 3D connect-the-dots game
  - Place "dots" at 3D coordinates
  - Connect-the-dots to form 3D shapes
- Pyramid: an example



- Using coordinate order
  - Polygons have a front and back:
    - By default, only the front side of a polygon is rendered
    - A polygon's winding order determines which side is the front
  - Use the right-hand rule:
    - Curl your right-hand fingers around the polygon perimeter in the order vertices are given (counter-clockwise)
    - Your thumb sticks out the front of the polygon

5. GeometryArray class hierarchy

- All geometry types are derived from Geometry

Class Hierarchy

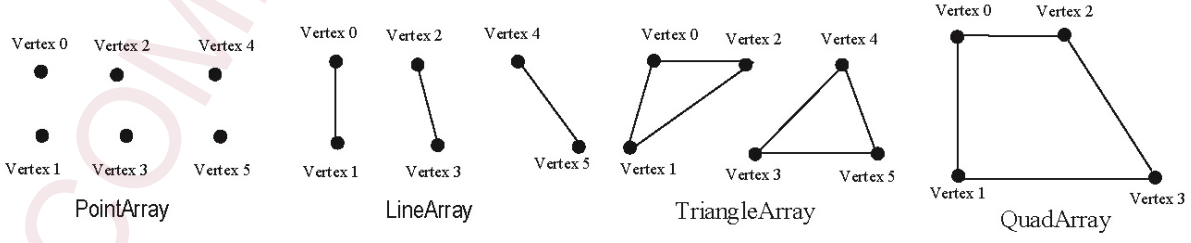
java.lang.Object
└ org.jogamp.java3d.SceneGraphObject
└ org.jogamp.java3d.NodeComponent
└ org.jogamp.java3d.Geometry
└ org.jogamp.java3d.GeometryArray

6. Building different types of geometry

- There are 14 types of geometry array in four groups:
  - Simple geometry:
    - `PointArray`, `LineArray`, `TriangleArray`, and `QuadArray`
  - Strip geometry:

    - `LineStripArray`, `TriangleStripArray`, and `TriangleFanArray`
  - Indexed simple geometry:
    - `IndexedPointArray`, `IndexedLineArray`, `IndexedTriangleArray`, and `IndexedQuadArray`
  - Indexed stripped geometry:
    - `IndexedLineStripArray`, `IndexedTriangleStripArray`, and `IndexedTriangleFanArray`

7. Building simple geometry:



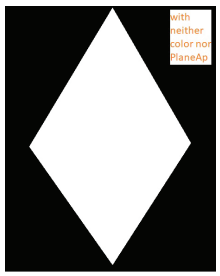
```
private Shape3D curveShape() {
    int n = 100;                                        // number of p
    float r = 1.8f, x, y, dx = r / n;                   // x distance
    PointArray pA = new PointArray(n,                   // declare an
            PointArray.COLOR_3 |
            PointArray.COORDINATES);

    Point3f pt = null;
    for (int i = 0; i < n; i++) {
        x = -0.9f + dx * i;                             // calculate x
        y = x * x - 0.5f;
        pt = new Point3f(x, y , 0.0f);
        pA.setCoordinate(i, pt);                        // set coordi
        pA.setColor(i, fun.Lime);                       // set color (
    }

    return new Shape3D(pA);
}
```

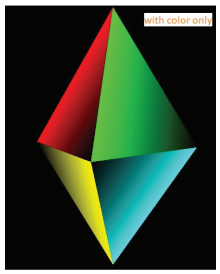
- A `PointArray` builds points with one point at each vertex
  - A *vertex* describes a polygon corner
    - It contains a required 3D coordinate
    - (optional) It may contain a color, a texture coordinate, and a lighting normal vector for appearance
  - Vertex normal must be defined to enable lighting for a surface geometry
    - For a given (polygonal) surface, its vertex normals define the surface's lighting condition while the coordinate winding order defines its front and back, which determines which side to draw
    - Lighting normals must be unit length

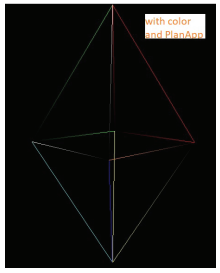
- A `LineArray` builds lines connecting pairs of vertices

```
private Shape3D lineShape() {
    float r = 0.6f, x, y;                                  // vertex a
    Point3f coor[] = new Point3f[5];

    LineArray lineArr = new LineArray(10,
            LineArray.COLOR_3 |
            LineArray.COORDINATES);
    for (int i = 0; i <= 4; i++) {
        x = (float) Math.cos(Math.PI / 180 * (90 + 72 * i)) * r;
        y = (float) Math.sin(Math.PI / 180 * (90 + 72 * i)) * r;
        coor[i] = new Point3f(x, y, 0.0f);
    }

    for (int i = 0; i <= 4; i++) {
        lineArr.setCoordinate(i * 2, coor[i]);
        lineArr.setCoordinate(i * 2 + 1, coor[(i + 2) % 5]);
        lineArr.setColor(i * 2, fun.Red);
        lineArr.setColor(i * 2 + 1, fun.Green);
    }

    return new Shape3D(lineArr);
}
```


```
private Geometry Tri1Geometry(Color3f clr) {
    TriangleArray Tri1 = new TriangleArray(3,
            TriangleArray.COLOR_3 |
            TriangleArray.COORDINATES);
    for (int i = 0; i < 2; i++)
        Tri1.setColor(i, clr);
    Tri1.setCoordinate(0, new Point3f(0.0f, 0.8f, 0.0f));
    Tri1.setCoordinate(1, new Point3f(0.0f, 0.0f, 0.5f));
    Tri1.setCoordinate(2, new Point3f(0.5f, 0.0f, 0.0f));
    return Tri1; }

private Shape3D triangleShape() {
    Shape3D shape3D = new Shape3D();
    shape3D.setAppearance(fun.geoPlaneApp(fun.Lime));

    shape3D.setGeometry(Tri1Geometry(fun.Red));
    shape3D.addGeometry(Tri2Geometry(fun.Green));
    shape3D.addGeometry(Tri3Geometry(fun.White));
    shape3D.addGeometry(Tri4Geometry(fun.Orange));
    shape3D.addGeometry(Tri5Geometry(fun.Yellow));
    shape3D.addGeometry(Tri6Geometry(fun.Cyan));
    shape3D.addGeometry(Tri7Geometry(fun.Blue));
    shape3D.addGeometry(Tri8Geometry(fun.Magenta));
    return shape3D;
    }
```

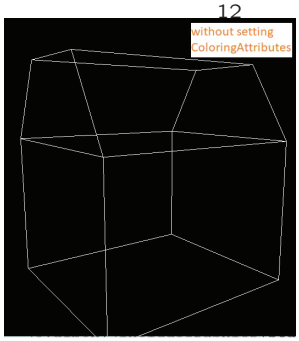


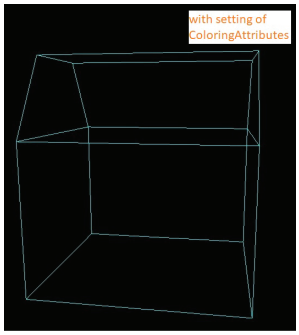




```
private Shape3D quadShape() {
    Vector3f topleft = new Vector3f(-0.22f, 0.55f, 0.45f);
    Vector3f midleft = new Vector3f(-0.45f, 0.17f, 0.45f);
    Vector3f botleft = new Vector3f(-0.45f, -0.55f, 0.45f);
    Vector3f topright = new Vector3f(0.22f, 0.55f, 0.45f);
    Vector3f midright = new Vector3f(0.45f, 0.17f, 0.45f);
    Vector3f botright = new Vector3f(0.45f, -0.55f, 0.45f);
    Vector3f btopleft = new Vector3f(-0.22f, 0.55f, -0.55f);
    Vector3f bmidleft = new Vector3f(-0.45f, 0.17f, -0.55f);
    Vector3f bbotleft = new Vector3f(-0.45f, -0.55f, -0.55f);
    Vector3f btopright = new Vector3f(0.22f, 0.55f, -0.55f);
    Vector3f bmidright = new Vector3f(0.45f, 0.17f, -0.55f);
    Vector3f bbotright = new Vector3f(0.45f, -0.55f, -0.55f);

    Point3f[] coords = {                // store the coordinates of the vertices
            new Point3f(topleft),  new Point3f(midleft),  new Point3f(midright),
            new Point3f(topright), new Point3f(btopleft), new Point3f(bmidleft),
            new Point3f(bmidright),new Point3f(btopright),new Point3f(botleft),
            new Point3f(botright), new Point3f(bbotright),new Point3f(bbotleft),
            new Point3f(midleft),  new Point3f(botleft),  new Point3f(botright),
            new Point3f(midright), new Point3f(btopleft), new Point3f(bmidleft),
            new Point3f(midleft), new Point3f(topleft),   new Point3f(topright),
            new Point3f(midright), new Point3f(bmidright), new Point3f(btopright),
            new Point3f(bmidleft), new Point3f(bbotleft), new Point3f(bbotright),
            new Point3f(bmidright) };

    QuadArray House_struct = new QuadArray(coords.length, QuadArray.COORDINATES);
    House_struct.setCoordinates(0, coords);

    return new Shape3D(House_struct, fun.geoPlaneApp(fun.Lime));
}
```

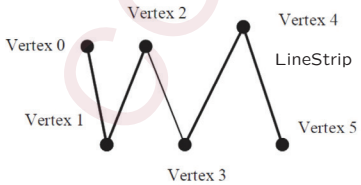

- A `TriangleArray`/`QuadArray` builds `triangles`/`quadrilaterals` between each *triple*/*quadruple* of vertices.
  - controlled by appearance attributes when rendered as surface

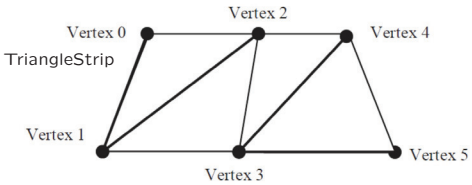


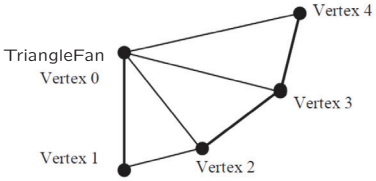



```
public Appearance geoPlaneApp(Color3f clr) {
    Appearance app = new Appearance();
    PolygonAttributes pa = new PolygonAttributes();
    pa.setPolygonMode(PolygonAttributes.POLYGON_LINE);
    pa.setCullFace(PolygonAttributes.CULL_NONE);
    app.setPolygonAttributes(pa);
    ColoringAttributes ca = new ColoringAttributes(clr,
            ColoringAttributes.FASTEST);
    app.setColoringAttributes(ca);
    return app; }
```

8. Building strip geometry

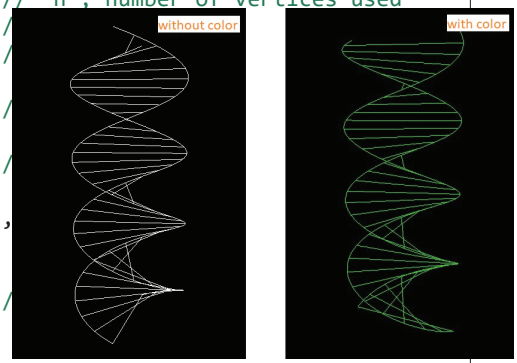




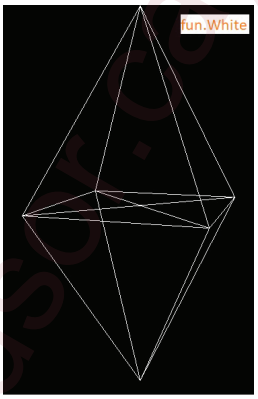


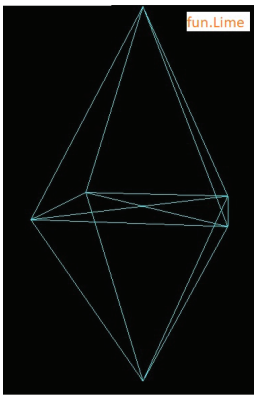
- Difference between *simple* and *strip* geometry
  - Simple geometry types use vertices in pairs, triples, and quadruples to build lines, triangles, and quadrilaterals one at a time
  - Strip geometry uses multiple vertices in a chain to build multiple lines and triangles by
    - providing a coordinate list (as always)
    - optionally providing lighting normal, color, and texture coordinate lists when rendering as surfaces, and
    - providing a strip length list, with each list entry giving the number of consecutive vertices to chain together


```
private Shape3D lineStripShape() {
    int i, j, n = 360;                                  // 'n', number of vertices used
    double u, r = 0.5;                                  //
                                                        //
    int[] strip = new int[2 + n / 2];                   //
    strip[0] = strip[1] = n;                            //
    for (int k = 2; k < (2 + n / 2); k++)               //
        strip[k] = 2;                                   //

    LineStripArray lsa = new LineStripArray((2 * n + n),
            LineStripArray.COLOR_3 |
            GeometryArray.COORDINATES, strip);
    for (i = 0; i < (2 * n + n); i++)                   //
        lsa.setColor(i, fun.Lime);

    for (i = 0; i < 2; i++, r = (i < 1) ? r : -r )      // for each of the two helix
        for (u = 0, j = 0; j < n; u = u + (720 / n), j++)
            lsa.setCoordinate(j + i * n, new Point3d(r * Math.cos(u * Math.PI / 180),
                    (-1.25 + (u * Math.PI / 180) / 5.0), r * Math.sin(u * Math.PI / 180)));

    for (u = 0, j = 0; j < n && u < 720; u = u + (10 * 720 / n)) {
        lsa.setCoordinate(j + (2 * n), new Point3d( r * Math.cos(u * Math.PI / 180),
                (-1.25 + (u * Math.PI / 180) / 5.0),  r * Math.sin(u * Math.PI / 180)));
        j++;
        lsa.setCoordinate(j + (2 * n), new Point3d( -r * Math.cos(u * Math.PI / 180),
                (-1.25 + (u * Math.PI / 180) / 5.0),  -r * Math.sin(u * Math.PI / 180)));
        j++;
    }                                                   // connecting the helix together
    return new Shape3D(lsa);
}
```

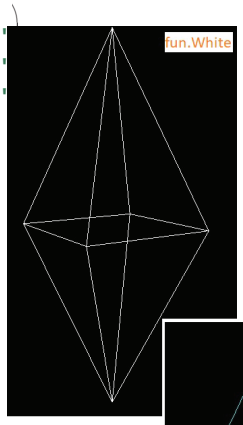


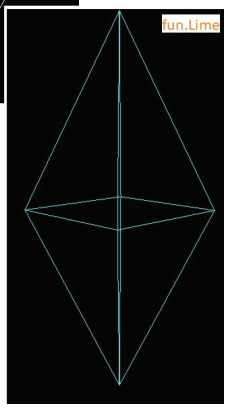
```
private Geometry Strip1Geometry(int n, int[] vC) { // Top 1
    TriangleStripArray Triangle = new TriangleStripArray(n,
            TriangleStripArray.COORDINATES, vC);
    Triangle.setCoordinate(0, new Point3f(0.0f, 0.0f, 0.5f));
    Triangle.setCoordinate(1, new Point3f(0.0f, 0.8f, 0.0f));
    Triangle.setCoordinate(2, new Point3f(0.5f, 0.0f, 0.0f));
    Triangle.setCoordinate(3, new Point3f(0.0f, 0.0f, -0.5f));
    return Triangle; }
private Geometry Strip2Geometry(int n, int[] vC) { // Top 2
private Geometry Strip3Geometry(int n, int[] vC) { // Bottom 3
private Geometry Strip4Geometry(int n, int[] vC) { // Bottom 4

private Shape3D triangleStripShape() {
    int n = 4;
    int vC[] = { n };

    Shape3D shape3D = new Shape3D(); // create an empty shape
    shape3D.setGeometry(Strip1Geometry(n, vC));
    shape3D.addGeometry(Strip2Geometry(n, vC));
    shape3D.addGeometry(Strip3Geometry(n, vC));
    shape3D.addGeometry(Strip4Geometry(n, vC));

    shape3D.setAppearance(fun.geoPlaneApp(fun.Lime));

    return shape3D;
}
```






```
private Shape3D triangleFanShape() {
    int n, N = 5;                                       // 'N'
    int m = 2 * (N+1), StripCount[] = { N+1, N+1 };     // 'm'
    float a, x, z, r = 0.4f;                            // 'r'
    Point3f coords[] = new Point3f[m];

    coords[0 * (N + 1)] = new Point3f(0.0f, 0.8f, 0.0f);
    coords[1 * (N + 1)] = new Point3f(0.0f, -0.8f, 0.0f);

    for (a = 0, n = 0; n < N; n++) {
        a = (float) (2.0 * Math.PI / (N - 1) * n);
        x = (float) (r * Math.cos(a));
        z = (float) (r * Math.sin(a));
        coords[0 * (N + 1) + N - n] = new Point3f(x, 0f, z);
        coords[1 * (N + 1) + n + 1] = new Point3f(x, 0f, z);
    }
    TriangleFanArray tfa = new TriangleFanArray(m,
            TriangleFanArray.COORDINATES, StripCount);
    tfa.setCoordinates(0, coords);

    return new Shape3D(tfa, fun.geoPlaneApp(fun.Lime));
}
```

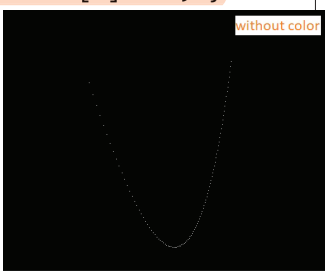


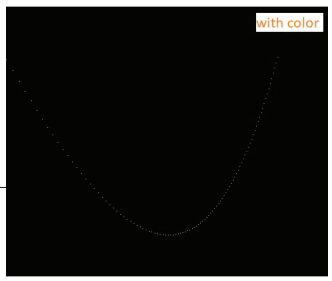


9. Building indexed simple geometry

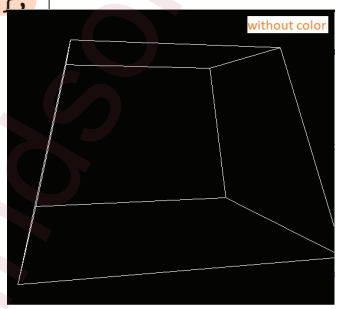
- For surfaces, the same vertices are used for adjacent lines and triangles
  - Simple and strip geometry require *redundant* coordinates, lighting normals, colors, and texture coordinates
- Indexed geometry uses indices along with the usual lists of coordinates, lighting normals, etc.
  - Indices select coordinates to use from your list
  - Use a coordinate multiple times, but give it only once
  - Indices also used for lighting normals, colors, and texture coordinates

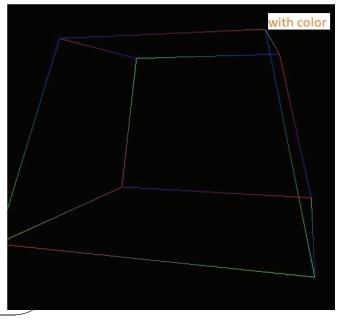

```
private Shape3D indexedPointShape() {
    int i, n = 100;                                          // number of points
    float r = 1.8f, x, y, dx = r / n;
    int[] colorIndices = new int[n], ptIndices = new int[n];

    IndexedPointArray array = new IndexedPointArray(n,
            PointArray.COLOR_3 | PointArray.COORDINATES , n);
    Color3f[] clr = new Color3f[n];
    for (i = 0; i < n; i++) { clr[i] = fun.Lime; colorIndices[i] = i; }
    array.setColors(0, clr);
    array.setColorIndices(0, colorIndices);

    Point3f[] pts = new Point3f[n];
    for (i = 0; i < n; i++) {
        x = -0.9f + dx * i;
        y = x * x - 0.5f;
        pts[i] = new Point3f(x, y, 0.0f);
        ptIndices[i] = i;
    }
    array.setCoordinates(0, pts);
    array.setCoordinateIndices(0, ptIndices);

    return new Shape3D(array);
}
```



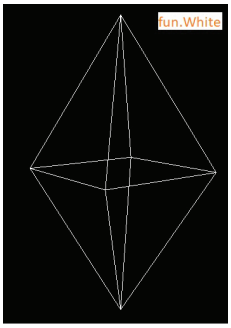


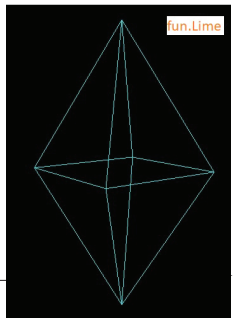
```
private Shape3D indexedLineShape() {
    IndexedLineArray array = new IndexedLineArray(8,
            LineArray.COLOR_3 | LineArray.COORDINATES, 24);
    Color3f[] color = {fun.Red, fun.Blue, fun.Green};
    int[] colorIndices = { 0, 1, 2, 0, 1, 2, 0, 1, 2,
            0, 1, 2, 0, 1, 2, 0, 1, 2, 0, 1, 2, 0, 1, 2 };
    array.setColors(0, color);
    array.setColorIndices(0, colorIndices);

    Point3f[] pts = new Point3f[8];
    pts[0] = new Point3f(0.4f, 0.6f, 0.4f);
    pts[1] = new Point3f(-0.4f, 0.6f, 0.4f);
    pts[2] = new Point3f(0.4f, 0.6f, -0.4f);
    pts[3] = new Point3f(-0.4f, 0.6f, -0.4f);
    pts[4] = new Point3f(0.6f, -0.2f, 0.6f);
    pts[5] = new Point3f(-0.6f, -0.2f, 0.6f);
    pts[6] = new Point3f(0.6f, -0.2f, -0.6f);
    pts[7] = new Point3f(-0.6f, -0.2f, -0.6f);
    int[] indices = { 0, 1, 0, 2, 2, 3, 1, 3, 4, 5,
            4, 6, 6, 7, 5, 7, 0, 4, 2, 6, 3, 7, 1, 5 };
    array.setCoordinates(0, pts);
    array.setCoordinateIndices(0, indices);

    return new Shape3D(array);
}
```



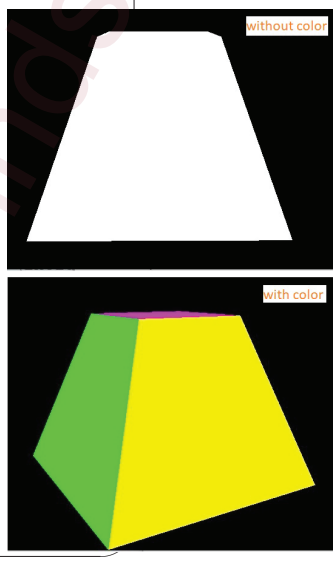



```
private Shape3D indexedTriangleShape() {
    IndexedTriangleArray Tri = new IndexedTriangleArray (6, GeometryArray.COORDINATES,24);
    Tri.setCoordinate(0, new Point3f(0f, 0.8f, 0f));
    Tri.setCoordinate(1, new Point3f(0.0f,0.0f,-0.5f));
    Tri.setCoordinate(2, new Point3f(0.5f, 0f, 0f));
    Tri.setCoordinate(3, new Point3f(-0.5f,0.0f,0.0f));
    Tri.setCoordinate(4, new Point3f(0f, 0f, 0.5f));
    Tri.setCoordinate(5, new Point3f(0f, -0.8f, 0.0f));

    //Top Diamond
    Tri.setCoordinateIndex( 0, 0 ); Tri.setCoordinateIndex( 1, 4 );
    Tri.setCoordinateIndex( 2, 2 ); Tri.setCoordinateIndex( 3, 0 );
    Tri.setCoordinateIndex( 4, 2 ); Tri.setCoordinateIndex( 5, 1 );
    Tri.setCoordinateIndex( 6, 0 ); Tri.setCoordinateIndex( 7, 1 );
    Tri.setCoordinateIndex( 8, 3 ); Tri.setCoordinateIndex( 9, 0 );
    Tri.setCoordinateIndex( 10,3 ); Tri.setCoordinateIndex( 11,4 );
    //Bottom Diamond
    Tri.setCoordinateIndex( 12,5 ); Tri.setCoordinateIndex( 13,4 );
    Tri.setCoordinateIndex( 14,2 ); Tri.setCoordinateIndex( 15,5 );
    Tri.setCoordinateIndex( 16,2 ); Tri.setCoordinateIndex( 17,1 );
    Tri.setCoordinateIndex( 18,5 ); Tri.setCoordinateIndex( 19,1 );
    Tri.setCoordinateIndex( 20,3 ); Tri.setCoordinateIndex( 21,5 );
    Tri.setCoordinateIndex( 22,3 ); Tri.setCoordinateIndex( 23,4 );

    return new Shape3D(Tri, fun.geoPlaneApp(fun.Lime));
}
```



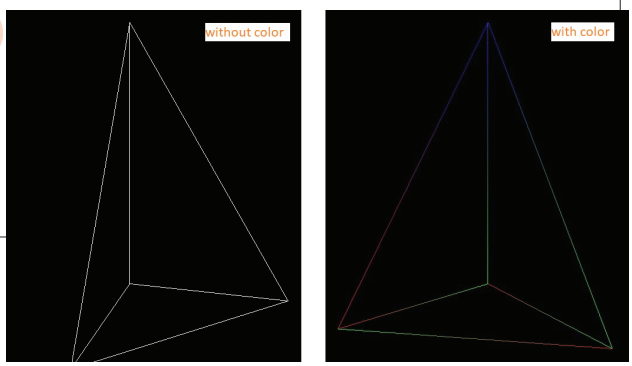


```
private Shape3D indexedQuadShape() {
    IndexedQuadArray array = new IndexedQuadArray(8,
            QuadArray.COLOR_3 | QuadArray.COORDINATES, 24);
    Color3f[] color = {fun.Magenta, fun.Red, fun.Green, fun.Blue,
            fun.Cyan, fun.Yellow, fun.Orange, fun.White};
    int[] colorIndices = { 0, 0, 0, 0, 1, 1, 1, 1, 2,
            2, 2, 2, 3, 3, 3, 3, 4, 4, 4, 4, 5, 5, 5, 5 };
    array.setColors(0, color);
    array.setColorIndices(0, colorIndices);

    Point3f[] pts = new Point3f[8];
    pts[0] = new Point3f(0.25f, 0.25f, 0.25f);
    pts[1] = new Point3f(-0.25f, 0.25f, 0.25f);
    pts[2] = new Point3f(0.25f, 0.25f, -0.25f);
    pts[3] = new Point3f(-0.25f, 0.25f, -0.25f);
    pts[4] = new Point3f(0.5f, -0.5f, 0.5f);
    pts[5] = new Point3f(-0.5f, -0.5f, 0.5f);
    pts[6] = new Point3f(0.5f, -0.5f, -0.5f);
    pts[7] = new Point3f(-0.5f, -0.5f, -0.5f);
    int[] indices = { 0, 2, 3, 1, 4, 5, 7, 6, 0, 4,
            6, 2, 1, 3, 7, 5, 1, 5, 4, 0, 2, 6, 7, 3 };
    array.setCoordinates(0, pts);
    array.setCoordinateIndices(0, indices);

    return new Shape3D(array);
}
```

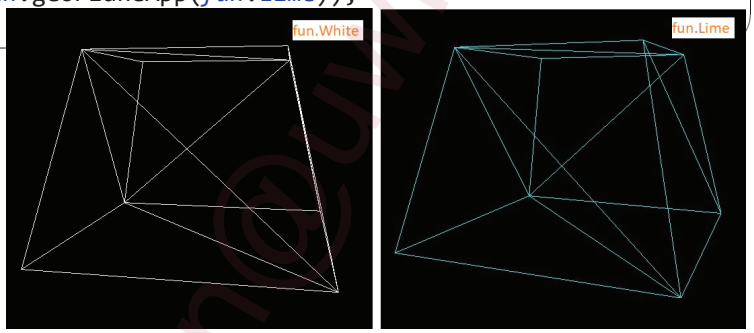


10. Building Indexed stripped geometry

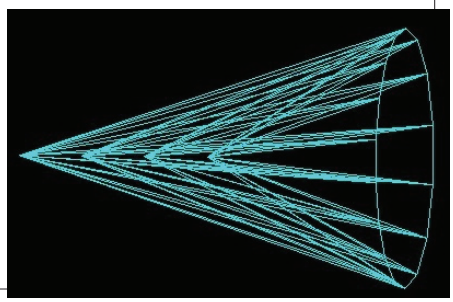
```
private Shape3D indexedLineStripShape() {
    Point3f[] pts = {new Point3f(0.0f, -0.5f, 0.0f), new Point3f(0.75f, -0.5f, 0.0f),
            new Point3f(0.0f, 0.75f, 0.0f), new Point3f(0.0f, -0.5f, 0.75f)};
    int[] indices = { 0, 1, 2, 3, 0, 2, 1, 3 };
    int strip[] = {6, 2};                    // declare with initialization

    IndexedLineStripArray ilsa = new IndexedLineStripArray(4,
            GeometryArray.COLOR_3 | GeometryArray.COORDINATES, 8, strip);
    Color3f[] color = {fun.Red, fun.Green, fun.Blue};
    int[] colorIndices = { 0, 1, 2, 0, 1, 2, 0, 1};
    ilsa.setColors(0, color);
    ilsa.setColorIndices(0, colorIndices);

    ilsa.setCoordinates(0, pts);
    ilsa.setCoordinateIndices(0, indices);

    return new Shape3D(ilsa);
}
```



```
private Shape3D indexedTriangleStripShape() {

    Point3f[] pts = {new Point3f(0.5f, 0.5f, 0.0f), new Point3f(0.0f, 0.5f, -0.5f),
            new Point3f(-0.5f, 0.5f, 0.0f), new Point3f(0.0f, 0.5f, 0.5f),
            new Point3f(0.75f, -0.25f, 0.0f), new Point3f(0.0f, -0.25f, -0.75f),
            new Point3f(-0.75f, -0.25f, 0.0f), new Point3f(0.0f, -0.25f, 0.75f) };
    int[] indices = { 0, 1, 3, 2, 6, 1, 5, 0, 4, 3, 7, 6, 4, 5 };
    int[] strip = { 14 };

    IndexedTriangleStripArray itsa = new IndexedTriangleStripArray(8,
            GeometryArray.COORDINATES, 14, strip);

    itsa.setCoordinates(0, pts);
    itsa.setCoordinateIndices(0, indices);

    return new Shape3D(itsa, fun.geoPlaneApp(fun.Lime));
}
```

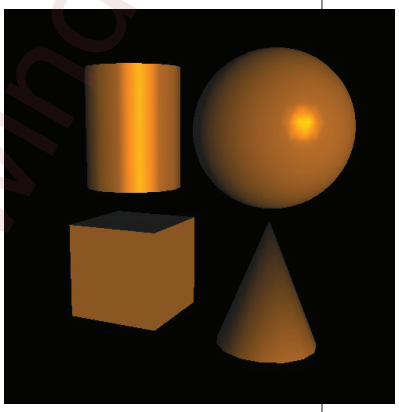



```
private Shape3D indexedTriangleFanShape() {
    Point3f[] fourPts = {new Point3f(0.0f, 0.0f, 0.8f), new Point3f(0.0f, 0.0f, 0.5f),
            new Point3f(0.0f, 0.0f, 0.2f), new Point3f(0.0f, 0.0f, -0.1f) };
    int i, j, n, m, N = 17, M = 4 * (N + 1);
    int stripCounts[] = { N + 1, N + 1, N + 1, N + 1 };
    float x, y, r = 0.6f;  double a;

    IndexedTriangleFanArray itfa = new IndexedTriangleFanArray(20,
                    IndexedTriangleFanArray.COORDINATES, M, stripCounts);
    itfa.setCoordinates(16, fourPts);       // set the central points
    for (a = 0, n = 0, m = 0; n < 17 && m < 16; a = 2.0 * Math.PI / (N - 1) * ++n, m++) {
        x = (float) (r * Math.cos(a)); y = (float) (r * Math.sin(a));
        itfa.setCoordinate(m, new Point3f(x, y, -1));
    }
    for (int k = 0; k < 4; k++) {
        itfa.setCoordinateIndex(k * 18, 16 + k);
        itfa.setCoordinateIndex(k * 18 + 17, 0);
        for (i = 1, j = k * 18 + 1; i < 17; i++, j++)
            itfa.setCoordinateIndex(j, i - 1);
    }
    return new Shape3D(itfa, fun.geoPlaneApp(fun.Lime));
}
```



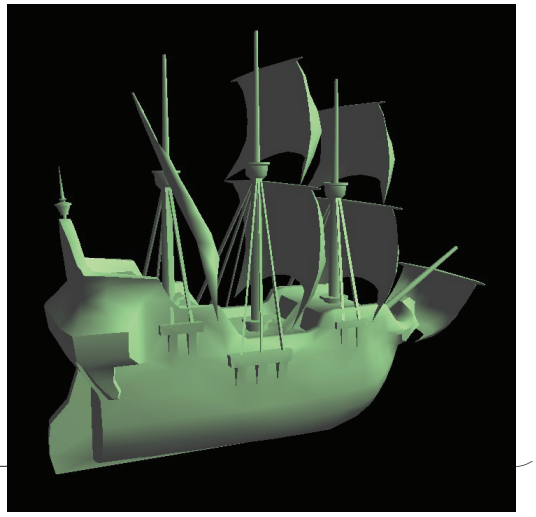
11. Using primitives: Box, Cone, Cylinder, Sphere

```
private static BranchGroup fourPrimitiveShape() {
    Vector3f[] position = {new Vector3f(0.3f, 0.3f, -0.3f), new Vector3f(-0.3f, 0.3f, -0.3f),
            new Vector3f(-0.3f, -0.3f, -0.3f), new Vector3f(0.3f, -0.3f, -0.3f)};
    BranchGroup objBG = new BranchGroup();

    DirectionalLight light = new DirectionalLight(CommonsXY.Orange, new Vector3f(-1f, 0f, -1f));
    light.setInfluencingBounds(new BoundingSphere(new Point3d(), 1000d));
    objBG.addChild(light);

    Appearance app = new Appearance();
    app.setMaterial(AppearanceExtra.setMaterial(CommonsXY.White));

    Primitive[] priShape = new Primitive[4];
    priShape[0] = new Sphere(0.3f, Sphere.GENERATE_NORMALS, 60, app);
    priShape[1] = new Cylinder(0.2f, 0.5f, app);
    priShape[2] = new Box(0.2f, 0.2f, 0.2f, app);
    priShape[3] = new Cone(0.2f, 0.5f, app);

    Transform3D trans3d = new Transform3D();
    TransformGroup trans = null;
    for (int i = 0; i < 4; i++) {
        trans3d.setTranslation(position[i]);
        trans = new TransformGroup(trans3d);
        objBG.addChild(trans);
        trans.addChild(priShape[i]);
    }
    return objBG;
}
```



12. Loading external object

```
private BranchGroup loadShape() {
    int flags = ObjectFile.RESIZE | ObjectFile.TRIANGULATE | ObjectFile.STRIPIFY;
    ObjectFile f = new ObjectFile(flags, (float) (60 * Math.PI / 180.0));
    Scene s = null;
    try {
        s = f.load("images/galleon.obj");
    } catch (FileNotFoundException e) {
        System.err.println(e);
        System.exit(1);
    } catch (ParsingErrorException e) {
        System.err.println(e);
        System.exit(1);
    } catch (IncorrectFormatException e) {
        System.err.println(e);
        System.exit(1);
    }

    return s.getSceneGroup();
}
```



13. Loading 3D fonts

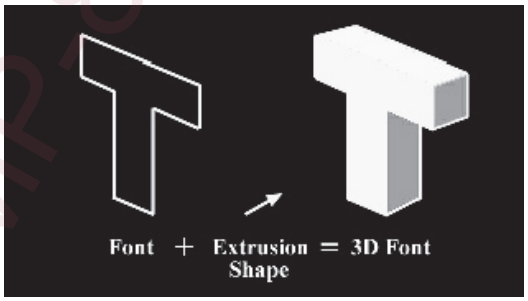
```
public static TransformGroup letters3D(String textString, double s, Point3f p) {
                        // font name, font style, font size
    Font my2DFont = new Font("Arial", Font.PLAIN, 1);
    FontExtrusion myExtrude = new FontExtrusion( );
    Font3D font3D = new Font3D( my2DFont, myExtrude );

    Text3D text3D = new Text3D(font3D, textString, p);
    Appearance app = new Appearance();
    app.setMaterial(AppearanceExtra.setMaterial());

    Transform3D scaler = new Transform3D();
    scaler.setScale(s);            // scaling 4x4 matrix
    TransformGroup scene_TG = new TransformGroup(scaler);
    scene_TG.addChild(new Shape3D(text3D, app));

    return scene_TG;
}
```

- Steps to build 3D text

(1) Select a 2D font with `java.awt.Font`

(2) Describe a 2D extrusion shape with java.awt.Sahpe in a `FontExtrusion`

(3) Create a 3D font by extruding the 2D font along the extrusion shape with a `Font3D`

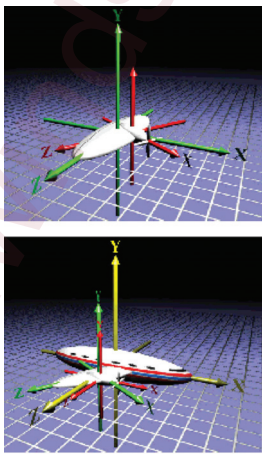


(4) Create 3D text using a string and a `Font3D` in a `Text3D`

PART II. TRANSFORMING SHAPES

1. The default location of 3D shapes

- By default, all shapes are built within a shared *world coordinate system*
- A `TransformGroup` builds a new coordinate system for its children, relative to its parent
  - *Translate* to change position
  - *Rotate* to change orientation
  - *Scale* to change size
  - Use in combination
- Transforms can be nested to include one `TransformGroup` within another



- A 3D transform must be *affine*
  - The transformation must preserve lines and parallelism (but not necessarily distances and angles).
- A 3D transform must be congruent if used in a `TransformGroup` before attaching a `ViewPlatform`
  - No non-uniform scaling of the viewpoint

2. 3D transforms are internally represented as a 4×4 matrix

- A 3D point $\mathbf{P}$ is transformed to $\mathbf{P}'$ by τ, i.e., $\mathbf{P}' = \tau\mathbf{P}$:

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & m_{11} & m_{12} & m_{13} \\ m_{20} & m_{21} & m_{22} & m_{23} \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

- Translation along three axes

$$\begin{aligned}\mathbf{T}_{xyz}(\delta_x, \delta_y, \delta_z) &= \mathbf{T}_x(\delta_x)\mathbf{T}_y(\delta_y)\mathbf{T}_z(\delta_z) \\ &= \begin{bmatrix} 1 & 0 & 0 & \delta_x \\ 0 & 1 & 0 & \delta_y \\ 0 & 0 & 1 & \delta_z \\ 0 & 0 & 0 & 1 \end{bmatrix}\end{aligned}$$

- Scaling on three axes

$$\begin{aligned}\mathbf{S}_{xyz}(s_1, s_2, s_3) &= \mathbf{S}_x(s_1)\mathbf{S}_y(s_2)\mathbf{S}_z(s_3) \\ &= \begin{bmatrix} s_1 & 0 & 0 & 0 \\ 0 & s_2 & 0 & 0 \\ 0 & 0 & s_3 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}\end{aligned}$$

  - Uniform scaling: $s_1 = s_2 = s_3$

- Rotation around three axes or a unit vector
  - rotate around $\mathbf{x}$ for α

$$\mathbf{R}_{\mathbf{x}}(\alpha) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\alpha & -\sin\alpha & 0 \\ 0 & \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

  - rotate around $\mathbf{y}$ for β

$$\mathbf{R}_{\mathbf{y}}(\beta) = \begin{bmatrix} \cos\beta & 0 & \sin\beta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\beta & 0 & \cos\beta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- rotate around $\mathbf{z}$ for γ

$$\mathbf{R_z}(\gamma) = \begin{bmatrix} \cos\gamma & -\sin\gamma & 0 & 0 \\ \sin\gamma & \cos\gamma & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

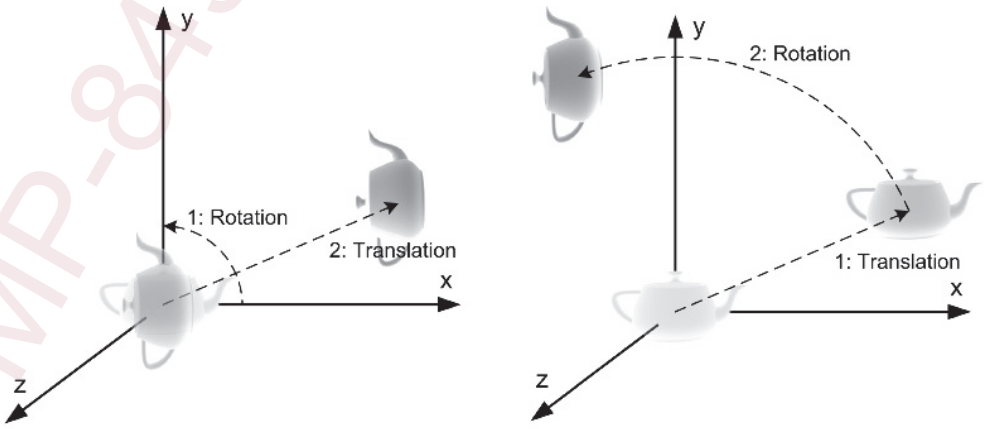
- rotate around a *unit* vector $\vec{\mathbf{V}}_k$ for θ

$$\mathbf{R}(\tilde{\mathbf{V}}_\mathbf{k}, \theta) = \begin{bmatrix} k_x k_x v\theta + c\theta & k_x k_y v\theta - k_z s\theta & k_x k_z v\theta + k_y s\theta & 0 \\ k_x k_y v\theta + k_z s\theta & k_y k_y v\theta + c\theta & k_y k_z v\theta - k_x s\theta & 0 \\ k_x k_z v\theta - k_y s\theta & k_y k_z v\theta + k_x s\theta & k_z k_z v\theta + c\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

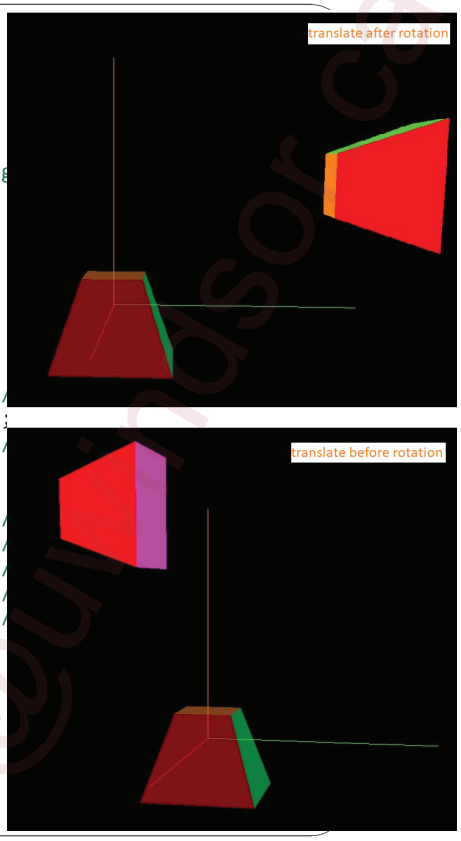
where, $c\theta = \cos\theta$, $s\theta = \sin\theta$, $v\theta = 1 - \cos\theta$, and

$$\vec{\mathbf{V}}_k = [k_x \;\; k_y \;\; k_z], \qquad \sqrt{k_x^2 + k_y^2 + k_z^2} = 1$$

- Due to non-commutativity of transformations composition, it is critical to understand the order in which transformation matrices are multiplied.



```
public class ShapeTransformation extends Applet {
    private static final long serialVersionUID = 1L;
    private static BasicFunctions fun = null;

    private Geometry indexedQuadCube() {
    private Shape3D transpCube() {
    /* a function to attach a right-hand frame in red (Z), g
    private Shape3D threeAxices() {
    /* a function to attach objects to scene_TG */
    private void createContent(TransformGroup scene_TG) {
        scene_TG.addChild(threeAxices());

        TransformGroup comp_TG = new TransformGroup();
        scene_TG.addChild(transpCube());

        Transform3D translator = new Transform3D();
        translator.setTranslation(new Vector3f(2f, 1f, 0f));
        Transform3D rotator = new Transform3D();
        rotator.rotZ(Math.PI / 2);

        Transform3D trfm = new Transform3D();
        trfm.mul(translator);
        trfm.mul(rotator);
        comp_TG.setTransform(trfm);
        comp_TG.addChild(new Shape3D(indexedQuadCube()));

        scene_TG.addChild(comp_TG);
    }
    /* a constructor to create content branch and attach to
    public ShapeTransformation() {
    /* the main entrance of the application */
    public static void main(String[] args) {
}
```



- Steps to transform a shape or group of shapes

(1) Build or obtain a shape, e.g., `Shape3D shape3D;`

(2) For each transformation, introduce a 4×4 `Transform3D` matrix and then specify the transformation, e.g.,

```
Transform3D rotator = new Transform3D( );
rotator.rotZ(Math.PI / 2 );        // 90 degrees
```

(3) Multiple each of the matrices, with the last one being the first transformation to apply to the shape, e.g.,

```
trfm.mul(translator); trfm.mul(rotator);
```

(4) Set a transform group to attach the shape, e.g.,

```
TransformGroup comp_TG = new TransformGroup( );
comp_TG.setTransform( trfm );
comp_TG.addChild( shape3D );
```

PART III. GROUPING SHAPES

1. Java3D manages lists of children nodes with different groups

   - Some simply group their children while others provide added functionality

Class Hierarchy

```
java.lang.Object
  └ org.jogamp.java3d.SceneGraphObject
      └ org.jogamp.java3d.Node
          └ org.jogamp.java3d.Group
              ├ org.jogamp.java3d.BranchGroup
              ├ org.jogamp.java3d.OrderedGroup
              │   └ org.jogamp.java3d.DecalGroup
              ├ org.jogamp.java3d.SharedGroup
              ├ org.jogamp.java3d.Switch
              └ org.jogamp.java3d.TransformGroup
```


2. Types of groups

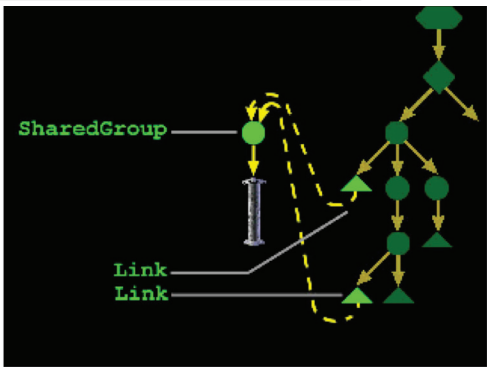
   - The most general-purpose is `Group`
     - It can have any number of children
     - Children can be added, inserted, removed, and "get" to/from a group
     - Child rendering order is determined by Java3D for better rendering efficiency
   - The most commonly used is `BranchGroup`
     - It can be attached to a `Locale` (or `SimpleUniverse`)
     - It can be compiled for optimization
     - It can be a child of any grouping node
     - It can be detached from its parent (if that parent has appropriate capabilities enabled)

- An `OrderedGroup` guarantees the rendering of children is in first-to-last order
- An `DecalGroup` renders children in first-to-last order
  - Children must be co-planar
  - All polygons must be facing the same way
  - First child is the underlying surface
  - The underlying surface must encompass all children – e.g., airport runway vs. markings of text/texture
- A `SharedGroup` creates a group of shapes that can be shared (used multiple times throughout a scene graph)
  - It is never added into the scene graph directly
  - It is referenced by one or more Link leaf nodes
  - It contains shapes and can be compiled prior "linking"

- `Link` extends `Leaf` to point to a `SharedGroup`

Class Hierarchy

```
java.lang.Object
  └ org.jogamp.java3d.SceneGraphObject
      └ org.jogamp.java3d.Node
          └ org.jogamp.java3d.Leaf
              └ org.jogamp.java3d.Link
```



- Steps to build `SharedGroup` with `Link`

(1) Build one or more shapes to share

```
Shape3D myShape = new Shape3D( myGeom, myAppear );
```

(2) Create a SharedGroup and add the shapes to it

```
SharedGroup myShared = new SharedGroup( );
myShared.addChild( myShape );
```

(3) Compile the SharedGroup for maximum performance

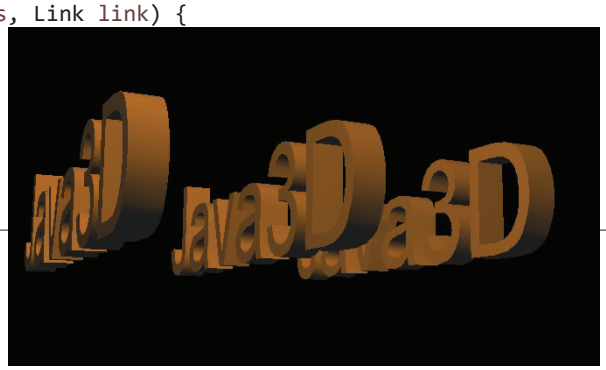
```
myShared.compile( );
```

(4) Use Link nodes to point to the group from another group

```
Link myLink = new Link( myShared );
TransformGroup myGroup = new TransformGroup( );
myGroup.addChild( myLink );
```



```
private static void createContent(TransformGroup scene_TG) {
    Vector3f[] post = {new Vector3f(0, 0, 1.2f), new Vector3f(0, 0, 0), new Vector3f(0, 0, -1.2f)};
    String str = "Java3D";
    SharedGroup shared3D = new SharedGroup( );
    shared3D.addChild( letters3D(str, new Point3f(-str.length() / 2.0f, -0.25f, 0f)) );
    shared3D.compile( );

    Transform3D scaler = new Transform3D();
    scaler.setScale(0.5d);      // scaling 4x4 matrix
    scene_TG.setTransform(scaler);
    Link[] links = new Link[3];
    for (int i = 0; i < 3; i++) {
        links[i] = new Link(shared3D);
        scene_TG.addChild(linked3D(post[i], links[i]));
    }
}
private static TransformGroup linked3D(Vector3f pos, Link link) {
    Transform3D position = new Transform3D();
    position.setTranslation(pos);
    TransformGroup posTG =
            new TransformGroup(position);
    posTG.addChild(link);
    return posTG;
}
```



3. `Switch` selects zero, one, or multiple children to render or process
   - Only selected children are rendered (shapes) or processed (lights, fog, backgrounds, behaviors)
     - Java3D decides the order to render children
   - Remember to enable the switch value to be changed while it is live or compiled
     - `setCapability( Switch.ALLOW_SWITCH_WRITE );`
   - Select which child to render by:
     - Passing its child index to `setWhichChild( )`
     - Or by passing in a special value:
       - Render no children: `CHILD_NONE`
       - Render all children: `CHILD_ALL`


```
private static void createContent(TransformGroup scene_TG) {
    String str = "Java3D";  float x_str = str.length() / 2.0f;
    String[] f_type = {"Arial", "SansSerif", "Monospaced"};
    int[] f_style = {Font.PLAIN, Font.ITALIC, Font.BOLD};
    Color3f[] f_clr = {CommonsXY.Lime, CommonsXY.Yellow, CommonsXY.Orange};
    Vector3f[] post = {new Vector3f(-x_str, 1f, 0), new Vector3f(-x_str, 0, 0),
            new Vector3f(-x_str, -1f, 0)};

    Switch mySwitch = new Switch( );
    mySwitch.setCapability( Switch.ALLOW_SWITCH_WRITE );
    for (int i = 0; i < 3; i++)
        mySwitch.addChild(letters3D(str, f_type[i], f_style[i], post[i], f_clr[i]));

    mySwitch.setWhichChild( 1 );          // 0 1 2 Switch.CHILD_ALL Switch.CHILD_NONE
    scene_TG.addChild(mySwitch);
}
private static TransformGroup letters3D(String txt, String tp, int stl, Vector3f pos, Color3f clr) {
    Transform3D position = new Transform3D();
    position.setTranslation(pos);
    TransformGroup posTG = new TransformGroup(position);

    Font my2DFont = new Font(tp, stl, 1);
    FontExtrusion myExtrude = new FontExtrusion( );
    Font3D font3D = new Font3D( my2DFont, myExtrude );
    Text3D text3D = new Text3D(font3D, txt);
    Appearance app = new Appearance();
    app.setMaterial(AppearanceExtra.setMaterial(clr));

    posTG.addChild(new Shape3D(text3D, app));
    return posTG;
}
```